

The practical way to deliver it is to generate it **one file (or one module) at a time** until the project is complete.

For example, I can provide in order:

1. hotel_management.sql
2. requirements.txt
3. config.py
4. database.py
5. login.py
6. main.py
7. dashboard.py
8. customer.py
9. room.py
10. booking.py
11. billing.py
12. employee.py
13. reports.py
14. invoice.py
15. README.md

Each file will be complete and ready to save, and together they will form the full application.

I can't honestly claim to provide the **entire** multi-file project in a single response because it exceeds the platform's message limits. If you want the full project, I'll generate every file sequentially until it's finished.

okay, go ahead

Excellent. We'll build it as a professional project.

Project Overview

Project Name: Hotel Management System

Language: Python 3

GUI: Tkinter + ttkbootstrap

Database: MySQL

Architecture: Object-Oriented Programming (MVC-style)

IDE: VS Code / PyCharm

Part 1

We'll create the database first.

Step 1. Install MySQL Connector

```
pip install mysql-connector-python
```

Install additional libraries:

```
pip install ttkbootstrap  
pip install pillow  
pip install reportlab
```

Step 2. Create Database

Create a file named:

```
hotel_management.sql
```

Paste the following SQL:

```
CREATE DATABASE IF NOT EXISTS hotel_management;  
  
USE hotel_management;
```

```
-- =====  
-- USERS  
-- =====
```

```
CREATE TABLE users(  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    username VARCHAR(50) UNIQUE,  
    password VARCHAR(255),  
    role VARCHAR(20)  
);
```

```
INSERT INTO users(username,password,role)  
VALUES  
('admin','admin123','Admin');
```

```
-- =====  
-- CUSTOMERS  
-- =====
```

```
CREATE TABLE customers(  
  
customer_id INT PRIMARY KEY AUTO_INCREMENT,  
  
customer_name VARCHAR(100),  
  
gender VARCHAR(20),  
  
mobile VARCHAR(20),  
  
email VARCHAR(100),  
  
address TEXT,  
  
id_proof VARCHAR(50),  
  
id_number VARCHAR(100)  
);
```

```
-- =====  
-- ROOM TYPES  
-- =====
```

```
CREATE TABLE room_types(  
  
type_id INT PRIMARY KEY AUTO_INCREMENT,
```

```

room_type VARCHAR(50),
price DECIMAL(10,2)
);

INSERT INTO room_types(room_type,price)
VALUES
('Standard',1500),
('Deluxe',2500),
('Super Deluxe',3500),
('Suite',6000);

-- =====
-- ROOMS
-- =====

CREATE TABLE rooms(
room_id INT PRIMARY KEY AUTO_INCREMENT,
room_number VARCHAR(10),
type_id INT,
status VARCHAR(20),
FOREIGN KEY(type_id)
REFERENCES room_types(type_id)
);

-- Sample Rooms

INSERT INTO rooms(room_number,type_id,status)
VALUES
('101',1,'Available'),
('102',1,'Available'),

```

```

('103',1,'Available'),
('201',2,'Available'),
('202',2,'Available'),
('203',2,'Available'),
('301',3,'Available'),
('302',3,'Available'),
('401',4,'Available');

-- =====
-- BOOKINGS
-- =====

CREATE TABLE bookings(

booking_id INT PRIMARY KEY AUTO_INCREMENT,

customer_id INT,

room_id INT,

check_in DATE,

check_out DATE,

adults INT,

children INT,

booking_date DATE,

status VARCHAR(20),

FOREIGN KEY(customer_id)

REFERENCES customers(customer_id),

FOREIGN KEY(room_id)

REFERENCES rooms(room_id)

```

```

);

-- =====
-- PAYMENTS
-- =====

CREATE TABLE payments(

payment_id INT PRIMARY KEY AUTO_INCREMENT,

booking_id INT,

amount DECIMAL(10,2),

gst DECIMAL(10,2),

total DECIMAL(10,2),

payment_mode VARCHAR(20),

payment_date DATE,

FOREIGN KEY(booking_id)

REFERENCES bookings(booking_id)

);

-- =====
-- EMPLOYEES
-- =====

CREATE TABLE employees(

employee_id INT PRIMARY KEY AUTO_INCREMENT,

employee_name VARCHAR(100),

designation VARCHAR(50),

salary DECIMAL(10,2),

mobile VARCHAR(20),

address TEXT

);

```

Step 3. Project Folder

HotelManagementSystem/

|

|— main.py

|— config.py

|— database.py

|— login.py

|— dashboard.py

|— customer.py

|— room.py

|— booking.py

|— billing.py

|— employee.py

|— reports.py

|— invoice.py

|— utils.py

|

|— assets/

| |— images/

| |— icons/

|

|— sql/

```
| └─ hotel_management.sql  
|  
└─ requirements.txt
```

Step 4. requirements.txt

```
mysql-connector-python
```

```
ttkbootstrap
```

```
Pillow
```

```
reportlab
```

Next (Part 2)

In the next part, we'll create the core Python files:

- `config.py`
- `database.py`
- `main.py`
- `login.py`

These will establish the MySQL connection, application entry point, and login system. After that, we'll build the dashboard and then each functional module (customers, rooms, bookings, billing, reports, etc.) until the application is complete.

please, go ahead for next step

Perfect. We'll continue with **Part 2**.

We'll create four files:

- `config.py`
- `database.py`

- main.py
- login.py

These files will establish the MySQL connection and provide a working login window.

config.py

Create a file named **config.py**.

```
# =====
# Database Configuration
# =====

DB_HOST = "localhost"
DB_USER = "root"
DB_PASSWORD = "1234"          # Change to your MySQL password
DB_NAME = "hotel_management"

# =====
# Application
# =====

APP_NAME = "Hotel Management System"
VERSION = "1.0"
```

database.py

```
import mysql.connector
from mysql.connector import Error
import config

class Database:

    def __init__(self):
        self.connection = None
        self.cursor = None

    def connect(self):

        try:
```

```

        self.connection = mysql.connector.connect(

            host=config.DB_HOST,
            user=config.DB_USER,
            password=config.DB_PASSWORD,
            database=config.DB_NAME

        )

        self.cursor =
self.connection.cursor(dictionary=True)

        print("Connected to MySQL")

    except Error as e:

        print("Database Error :", e)

    def execute(self, query, values=None):

        if values:
            self.cursor.execute(query, values)
        else:
            self.cursor.execute(query)

        self.connection.commit()

    def fetchall(self):

        return self.cursor.fetchall()

    def fetchone(self):

        return self.cursor.fetchone()

    def close(self):

        if self.connection:
            self.connection.close()

db = Database()
db.connect()

```

main.py

```
from login import Login

if __name__ == "__main__":

    Login()
```

login.py

```
import ttkbootstrap as ttk
from ttkbootstrap.constants import *

from tkinter import messagebox

from database import db

class Login:

    def __init__(self):

        self.window = ttk.Window(themename="flatly")

        self.window.title("Hotel Management System")

        self.window.geometry("500x400")

        self.window.resizable(False, False)

        title = ttk.Label(

            self.window,

            text="HOTEL MANAGEMENT SYSTEM",

            font=("Arial", 20, "bold")

        )

        title.pack(pady=20)

        ttk.Label(

            self.window,

            text="Username",
```

```

        font=("Arial", 12)

    ).pack()

    self.username = ttk.Entry(self.window, width=30)

    self.username.pack(pady=5)

    ttk.Label(

        self.window,

        text="Password",

        font=("Arial", 12)

    ).pack()

    self.password = ttk.Entry(

        self.window,

        width=30,

        show="*"

    )

    self.password.pack(pady=5)

    ttk.Button(

        self.window,

        text="Login",

        bootstyle=SUCCESS,

        command=self.login

    ).pack(pady=20)

    self.window.mainloop()

def login(self):

    username = self.username.get()

```

```
password = self.password.get()

sql = """

SELECT *

FROM users

WHERE username=%s

AND password=%s

"""

db.execute(sql, (username, password))

user = db.fetchone()

if user:

    messagebox.showinfo(

        "Success",

        "Login Successful"

    )

    self.window.destroy()

    from dashboard import Dashboard

    Dashboard(user)

else:

    messagebox.showerror(

        "Error",

        "Invalid Username or Password"

    )
```

Create a temporary `dashboard.py`

```
import ttkbootstrap as ttk

class Dashboard:

    def __init__(self, user):

        self.window = ttk.Window(themename="flatly")

        self.window.title("Dashboard")

        self.window.geometry("900x600")

        ttk.Label(

            self.window,

            text=f"Welcome {user['username']}",

            font=("Arial", 22, "bold")

        ).pack(pady=40)

        ttk.Label(

            self.window,

            text="Dashboard Coming Soon...",

            font=("Arial", 18)

        ).pack()

        self.window.mainloop()
```

Run the Project

`python main.py`

Use the credentials inserted by the SQL script:

- Username: `admin`
- Password: `admin123`

If everything is configured correctly, you should see the login window, and after a successful login, a simple dashboard placeholder.

One Important Improvement

Right now, the login compares **plain-text passwords**, which is **not secure**. In the next part, we'll replace this with **hashed passwords** using `bcrypt`, which is the recommended approach for real applications.

Part 3

Next we'll build a professional dashboard with:

- Sidebar navigation
- Customer Management
- Room Management
- Booking Management
- Billing
- Employees
- Reports
- Logout
- Live date and time
- Statistics cards (Available Rooms, Occupied Rooms, Today's Check-ins, Revenue)

From that point onward, each menu item will open a fully functional module until the entire hotel management system is complete.

Get smarter responses, upload files and images, and more.

Great. We will now create **Part 3: Professional Dashboard Module**.

This dashboard will provide:

- Modern `ttkbootstrap` GUI
- Sidebar navigation
- User information
- Current date/time
- Statistics cards:
 - Total Rooms
 - Available Rooms
 - Occupied Rooms
 - Total Customers
- Menu buttons for future modules
- Logout option

Before creating this file, update your project structure:

```
HotelManagementSystem/  
├── main.py  
├── config.py  
├── database.py  
├── login.py  
├── dashboard.py    <-- create this file now  
├── customer.py  
├── room.py  
├── booking.py  
├── billing.py  
├── employee.py  
└── reports.py
```

Replace `dashboard.py` with this code

```
import ttkbootstrap as ttk  
from ttkbootstrap.constants import *  
from tkinter import messagebox  
from datetime import datetime  
  
from database import db  
  
class Dashboard:
```



```

def __init__(self, user):

    self.user = user

    self.window = ttk.Window(
        themename="flatly"
    )

    self.window.title(
        "Hotel Management Dashboard"
    )

    self.window.geometry(
        "1200x700"
    )

    self.window.state("zoomed")

    # =====
    # Sidebar
    # =====

    self.sidebar = ttk.Frame(
        self.window,
        bootstyle="primary",
        width=220
    )

    self.sidebar.pack(
        side=LEFT,
        fill=Y
    )

    ttk.Label(
        self.sidebar,
        text="HOTEL SYSTEM",
        font=(
            "Arial",
            18,
            "bold"
        ),
        bootstyle="inverse-primary"
    ).pack(
        pady=30
    )

```

```
)
```

```
menu_buttons = [  
    ("Dashboard", self.home),  
    ("Customers", self.open_customer),  
    ("Rooms", self.open_rooms),  
    ("Bookings", self.open_booking),  
    ("Billing", self.open_billing),  
    ("Employees", self.open_employee),  
    ("Reports", self.open_reports)  
]
```

```
for text, command in menu_buttons:
```

```
    ttk.Button(  
        self.sidebar,  
        text=text,  
        width=20,  
        bootstyle="light",  
        command=command  
    ).pack(  
        pady=8,  
        padx=10  
    )
```

```
ttk.Button(  
    self.sidebar,  
    text="Logout",
```

```

        width=20,

        bootstyle="danger",

        command=self.logout

    ).pack(

        side=BOTTOM,

        pady=20

    )

# =====
# Main Area
# =====

self.main_frame = ttk.Frame(
    self.window

)

self.main_frame.pack(
    expand=True,
    fill=BOTH,
    padx=20,
    pady=20
)

self.home()

self.window.mainloop()

# =====
# Dashboard Home
# =====

def home(self):

```

```
for widget in self.main_frame.winfo_children():  
    widget.destroy()
```

```
ttk.Label(  
    self.main_frame,  
    text=f"Welcome, {self.user['username']}",  
    font=(  
        "Arial",  
        24,  
        "bold"  
    )  
) .pack(  
    anchor=W  
)
```

```
self.time_label = ttk.Label(  
    self.main_frame,  
    font=(  
        "Arial",  
        14  
    )  
)
```

```
self.time_label.pack(  
    anchor=W,  
    pady=10  
)
```

```
self.update_time()
```

```
# Cards
```

```
cards_frame = ttk.Frame(  

```

```
        self.main_frame
    )
    cards_frame.pack(
        pady=30
    )

    rooms = self.get_total_rooms()
    available = self.get_available_rooms()
    customers = self.get_customers()
    bookings = self.get_bookings()

    self.create_card(
        cards_frame,
        "Total Rooms",
        rooms,
        0
    )

    self.create_card(
        cards_frame,
        "Available Rooms",
        available,
        1
    )
```

```
self.create_card(  
    cards_frame,  
    "Customers",  
    customers,  
    2  
)
```

```
self.create_card(  
    cards_frame,  
    "Bookings",  
    bookings,  
    3  
)
```

```
# =====  
# Dashboard Cards  
# =====
```

```
def create_card(  
    self,  
    parent,  
    title,  
    value,  
    column):  
  
    frame = ttk.Frame(  
        parent,  
        bootstyle="info",
```

```
        width=220,

        height=130

    )

    frame.grid(

        row=0,

        column=column,

        padx=15

    )

    ttk.Label(

        frame,

        text=title,

        font=(

            "Arial",

            14,

            "bold"

        )

    ).pack(

        pady=15

    )

    ttk.Label(

        frame,

        text=str(value),

        font=(
```

```

        "Arial",

        25,

        "bold"

    )

).pack()

# =====
# Database Statistics
# =====

def get_total_rooms(self):

    db.execute(

        "SELECT COUNT(*) AS total FROM rooms"

    )

    return db.fetchone()["total"]

def get_available_rooms(self):

    db.execute(

        """
        SELECT COUNT(*) AS total
        FROM rooms
        WHERE status='Available'
        """

    )

    return db.fetchone()["total"]

def get_customers(self):

```



```

        db.execute(

            "SELECT COUNT(*) AS total FROM customers"

        )

        return db.fetchone()["total"]

def get_bookings(self):

    db.execute(

        "SELECT COUNT(*) AS total FROM bookings"

    )

    return db.fetchone()["total"]

# =====
# Clock
# =====

def update_time(self):

    now = datetime.now().strftime(

        "%d-%m-%Y    %I:%M:%S %p"

    )

    self.time_label.config(

        text=now

    )

    self.time_label.after(

        1000,

        self.update_time

```

```

    )

# =====
# Future Modules
# =====

def open_customer(self):

    messagebox.showinfo(
        "Customers",
        "Customer module will be connected in Part 4"
    )

def open_rooms(self):

    messagebox.showinfo(
        "Rooms",
        "Room module will be connected later"
    )

def open_booking(self):

    messagebox.showinfo(
        "Bookings",
        "Booking module will be connected later"
    )

def open_billing(self):

    messagebox.showinfo(
        "Billing",
        "Billing module will be connected later"
    )

def open_employee(self):

    messagebox.showinfo(
        "Employees",
        "Employee module will be connected later"
    )

```

```
)

def open_reports(self):

    messagebox.showinfo(
        "Reports",
        "Reports module will be connected later"
    )

def logout(self):

    self.window.destroy()

    from login import Login

    Login()
```

Test Dashboard

Run:

```
python main.py
```

Login:

```
Username : admin
Password : admin123
```

You should now see:

- Sidebar menu
- Dashboard cards
- Live clock
- Database statistics
- Logout button

Great. Now we will create **Part 4: Customer Management Module**.

This module will replace the placeholder customer button in the dashboard.

Features included:

- ✓ Add new customer
 - ✓ Update customer
 - ✓ Delete customer
 - ✓ Search customer
 - ✓ View all customers
 - ✓ Select customer from table
 - ✓ MySQL CRUD operations
 - ✓ Professional Tkinter GUI
-

Step 1: Create `customer.py`

Create a new file:

```
HotelManagementSystem/  
├── customer.py  <-- create this
```

Add the following code:

```
import ttkbootstrap as ttk  
  
from ttkbootstrap.constants import *  
  
from tkinter import messagebox  
  
from database import db  
  
  
class Customer:
```

```

def __init__(self):

    self.window = ttk.Toplevel()

    self.window.title(
        "Customer Management"
    )

    self.window.geometry(
        "1100x650"
    )

    # Variables

    self.customer_id = ttk.StringVar()

    self.name = ttk.StringVar()

    self.gender = ttk.StringVar()

    self.mobile = ttk.StringVar()

    self.email = ttk.StringVar()

    self.address = ttk.StringVar()

    self.id_proof = ttk.StringVar()

    self.id_number = ttk.StringVar()


    # =====
    # Title
    # =====

    ttk.Label(

        self.window,

        text="CUSTOMER MANAGEMENT",

        font=(
            "Arial",
            22,

```

```

        "bold"
    )

).pack(

    pady=15

)


# =====
# Form Frame
# =====

form = ttk.Frame(
    self.window
)

form.pack(
    padx=20,
    pady=10,
    fill=X
)


fields = [

    ("Name", self.name),

    ("Gender", self.gender),

    ("Mobile", self.mobile),

    ("Email", self.email),

    ("Address", self.address),

    ("ID Proof", self.id_proof),

    ("ID Number", self.id_number)

]

```

```
for i, (label, var) in enumerate(fields):
```

```
    ttk.Label(  
        form,  
        text=label  
    ).grid(  
        row=i//2,  
        column=(i%2)*2,  
        padx=10,  
        pady=8  
    )
```

```
    ttk.Entry(  
        form,  
        textvariable=var,  
        width=30  
    ).grid(  
        row=i//2,  
        column=(i%2)*2+1,  
        padx=10,  
        pady=8  
    )
```

```
# =====  
# Buttons  
# =====
```

```
btn_frame = ttk.Frame(  
    self.window  
)  
  
btn_frame.pack(  
    pady=10  
)  
  
ttk.Button(  
    btn_frame,  
    text="Add",  
    bootstyle="success",  
    command=self.add_customer  
) .grid(  
    row=0,  
    column=0,  
    padx=10  
)  
  
ttk.Button(  
    btn_frame,  
    text="Update",  
    bootstyle="primary",  
    command=self.update_customer  
) .grid(  
    row=0,  
    column=1,  
    padx=10  
)  
  
ttk.Button(  

```



```
        btn_frame,

        text="Delete",

        bootstyle="danger",

        command=self.delete_customer

    ).grid(
        row=0,
        column=2,
        padx=10
    )
```

```
ttk.Button(

    btn_frame,

    text="Clear",

    command=self.clear

).grid(
    row=0,
    column=3,
    padx=10
)
```

```
# =====
# Table
# =====
```

```
columns=(

    "ID",

    "Name",

    "Gender",

    "Mobile",

    "Email",
```

```

        "Address",
        "Proof",
        "Number"
    )

    self.table = ttk.Treeview(
        self.window,
        columns=columns,
        show="headings"
    )

    for col in columns:
        self.table.heading(
            col,
            text=col
        )
        self.table.column(
            col,
            width=120
        )

    self.table.pack(
        fill=BOTH,

```

```

        expand=True,

        padx=20,

        pady=20

    )

    self.table.bind(

        "<ButtonRelease-1>",

        self.select_customer

    )

    self.load_customers()


# =====
# Add Customer
# =====

def add_customer(self):

    if self.name.get()==" " or self.mobile.get()==" ":

        messagebox.showwarning(

            "Warning",

            "Name and Mobile are required"

        )

    return

sql = ""

INSERT INTO customers

```

```
(
customer_name,
gender,
mobile,
email,
address,
id_proof,
id_number
)
```

```
VALUES (%s,%s,%s,%s,%s,%s,%s)
```

```
"""
```

```
data=(
    self.name.get(),
    self.gender.get(),
    self.mobile.get(),
    self.email.get(),
    self.address.get(),
    self.id_proof.get(),
    self.id_number.get()
)
```

```
db.execute(
    sql,
    data
)
```

```
messagebox.showinfo(
```

```

        "Success",

        "Customer Added"

    )

    self.clear()

    self.load_customers()

# =====
# Load Customers
# =====

def load_customers(self):

    self.table.delete(

        *self.table.get_children()

    )

    db.execute(

        "SELECT * FROM customers"

    )

    rows=db.fetchall()

    for row in rows:

        self.table.insert(

            "",

            END,

            values=(

```

```

        row["customer_id"],
        row["customer_name"],
        row["gender"],
        row["mobile"],
        row["email"],
        row["address"],
        row["id_proof"],
        row["id_number"]

    )

)

```

```

# =====
# Select Customer
# =====

```

```

def select_customer(self,event):

    selected=self.table.focus()

    data=self.table.item(selected)

    if data["values"]:

        row=data["values"]

        self.customer_id.set(row[0])

        self.name.set(row[1])

        self.gender.set(row[2])

        self.mobile.set(row[3])

```

```
        self.email.set(row[4])

        self.address.set(row[5])

        self.id_proof.set(row[6])

        self.id_number.set(row[7])
```

```
# =====
# Update
# =====
```

```
def update_customer(self):
```

```
    sql="""

    UPDATE customers SET

    customer_name=%s,

    gender=%s,

    mobile=%s,

    email=%s,

    address=%s,

    id_proof=%s,

    id_number=%s

    WHERE customer_id=%s

    """
```

```
    data=(

        self.name.get(),

        self.gender.get(),
```

```

        self.mobile.get(),
        self.email.get(),
        self.address.get(),
        self.id_proof.get(),
        self.id_number.get(),
        self.customer_id.get()
    )

    db.execute(
        sql,
        data
    )

    messagebox.showinfo(
        "Success",
        "Customer Updated"
    )

    self.load_customers()

# =====
# Delete
# =====

def delete_customer(self):
    if self.customer_id.get()=="":

```



```

        return

    db.execute(

        """

        DELETE FROM customers

        WHERE customer_id=%s

        """,

        (

            self.customer_id.get(),

        )

    )

    messagebox.showinfo(

        "Deleted",

        "Customer Removed"

    )

    self.clear()

    self.load_customers()

# =====
# Clear
# =====

def clear(self):

    self.customer_id.set("")

```

```
self.name.set("")
self.gender.set("")
self.mobile.set("")
self.email.set("")
self.address.set("")
self.id_proof.set("")
self.id_number.set("")
```

Step 2: Connect Dashboard Button

Open `dashboard.py`.

Find:

```
def open_customer(self):
    messagebox.showinfo(
        "Customers",
        "Customer module will be connected in Part 4"
    )
```

Replace it with:

```
def open_customer(self):
    from customer import Customer
    Customer()
```

Test

Run:

```
python main.py
```

Login:

Username: admin
Password: admin123

Click:

Customers

You should now get:

- Customer entry form
- Add button
- Update button
- Delete button
- Customer table

Next: Part 5 — Room Management Module

We will add:

- ✓ Room list
- ✓ Room type management
- ✓ Room price
- ✓ Available/Occupied status
- ✓ Add/Edit/Delete rooms
- ✓ Room availability tracking
- ✓ Connection with booking system

Great. We will now create **Part 5: Room Management Module**.

This module will manage hotel rooms and will connect later with the booking system.

Features included:

- ✓ View all rooms
 - ✓ Add new room
 - ✓ Update room details
 - ✓ Delete room
 - ✓ Room type selection
 - ✓ Room price display
 - ✓ Room availability status
 - ✓ MySQL CRUD operations
-

Step 1: Create `room.py`

Create:

```
HotelManagementSystem/  
└─ room.py    <-- create this file
```

Add this code:

```
import tkinterbootstrap as ttk  
  
from tkinterbootstrap.constants import *  
  
from tkinter import messagebox  
  
from database import db  
  
class Room:  
  
    def __init__(self):  
        self.window = ttk.Toplevel()  
  
        self.window.title(  
            "Room Management"  
        )
```

```
self.window.geometry(  
    "1000x600"  
)  
  
# Variables  
  
self.room_id = ttk.StringVar()  
self.room_number = ttk.StringVar()  
self.room_type = ttk.StringVar()  
self.price = ttk.StringVar()  
self.status = ttk.StringVar()  
  
# Title  
  
ttk.Label(  
    self.window,  
    text="ROOM MANAGEMENT",  
    font=(  
        "Arial",  
        22,  
        "bold"  
    )  
) .pack(  
    pady=15  
)  
  
# Form Frame  
  
form = ttk.Frame(  
    self.window
```

```
)

form.pack(

    pady=10

)


# Room Number

ttk.Label(

    form,

    text="Room Number"

).grid(

    row=0,

    column=0,

    padx=10,

    pady=10

)


ttk.Entry(

    form,

    textvariable=self.room_number

).grid(

    row=0,

    column=1

)
```

```
# Room Type

ttk.Label(

    form,

    text="Room Type"

).grid(

    row=0,

    column=2,

    padx=10

)

self.type_box = ttk.Combobox(

    form,

    textvariable=self.room_type,

    values=[

        "Standard",

        "Deluxe",

        "Super Deluxe",

        "Suite"

    ]

)

self.type_box.grid(

    row=0,

    column=3
```

```
)
```

```
# Price
```

```
ttk.Label(
```

```
    form,
```

```
    text="Price"
```

```
).grid(
```

```
    row=1,
```

```
    column=0,
```

```
    pady=10
```

```
)
```

```
ttk.Entry(
```

```
    form,
```

```
    textvariable=self.price
```

```
).grid(
```

```
    row=1,
```

```
    column=1
```

```
)
```

```
# Status
```

```
ttk.Label(
```

```
    form,
```

```
    text="Status"
```



```
) .grid(
    row=1,
    column=2
)

ttk.Combobox(
    form,
    textvariable=self.status,
    values=[
        "Available",
        "Occupied",
        "Maintenance"
    ]
) .grid(
    row=1,
    column=3
)

# Buttons

buttons = ttk.Frame(
    self.window
)

buttons.pack(
    pady=10
```

```
)
```

```
ttk.Button(  
    buttons,  
    text="Add",  
    bootstyle="success",  
    command=self.add_room
```

```
).grid(  
    row=0,  
    column=0,  
    padx=10
```

```
)
```

```
ttk.Button(  
    buttons,  
    text="Update",  
    bootstyle="primary",  
    command=self.update_room
```

```
).grid(  
    row=0,  
    column=1,  
    padx=10
```

```
)
```

```
ttk.Button(  
    buttons,  
    text="Delete",  
    bootstyle="danger",  
    command=self.delete_room  
) .grid(  
    row=0,  
    column=2,  
    padx=10  
)
```

```
ttk.Button(  
    buttons,  
    text="Clear",  
    command=self.clear  
) .grid(  
    row=0,  
    column=3  
)
```

```
# Table
```

```
columns=(
```

```
        "ID",
        "Room No",
        "Type",
        "Price",
        "Status"
    )

    self.table=ttk.Treeview(
        self.window,
        columns=columns,
        show="headings"
    )

    for col in columns:

        self.table.heading(
            col,
            text=col
        )

        self.table.column(
            col,
            width=150
        )
```

```

        self.table.pack(

            fill=BOTH,

            expand=True,

            padx=20,

            pady=20

        )


        self.table.bind(

            "<ButtonRelease-1>",

            self.select_room

        )


        self.load_rooms()


# =====
# Add Room
# =====


def add_room(self):

    # Find Room Type ID


    db.execute(

        """

        SELECT type_id

        FROM room_types

```

```
WHERE room_type=%s
"""
(
    self.room_type.get(),
)
)
```

```
result=db.fetchone()
```

```
if result is None:
```

```
    messagebox.showerror(
        "Error",
        "Invalid Room Type"
    )
    return
```

```
db.execute(
    """
    INSERT INTO rooms
    (
        room_number,
        type_id,
        status
    )
```

```

VALUES(%s,%s,%s)

""",

(
    self.room_number.get(),
    result["type_id"],
    self.status.get()
)
)

messagebox.showinfo(
    "Success",
    "Room Added"
)

self.load_rooms()

self.clear()

# =====
# Load Rooms
# =====

def load_rooms(self):

    self.table.delete(
        *self.table.get_children()
    )

```

```

db.execute(
    """
    SELECT
        rooms.room_id,
        rooms.room_number,
        room_types.room_type,
        room_types.price,
        rooms.status

    FROM rooms

    JOIN room_types
    ON rooms.type_id=
        room_types.type_id
    """
)

rows=db.fetchall()

for row in rows:

    self.table.insert(
        "",
        END,

```



```

        values=(
            row["room_id"],
            row["room_number"],
            row["room_type"],
            row["price"],
            row["status"]
        )
    )

# =====
# Select Room
# =====

def select_room(self,event):

    selected=self.table.focus()

    data=self.table.item(selected)

    if data["values"]:

        row=data["values"]

        self.room_id.set(row[0])
        self.room_number.set(row[1])
        self.room_type.set(row[2])
        self.price.set(row[3])

```

```
self.status.set(row[4])
```

```
# =====  
# Update Room  
# =====
```

```
def update_room(self):
```

```
    db.execute(
```

```
        """
```

```
        SELECT type_id
```

```
        FROM room_types
```

```
        WHERE room_type=%s
```

```
        """,
```

```
        (
```

```
            self.room_type.get(),
```

```
        )
```

```
    )
```

```
    result=db.fetchone()
```

```
    db.execute(
```

```
        """
```

```
        UPDATE rooms
```

```
        SET
```

```
        room_number=%s,
```

```

        type_id=%s,

        status=%s

WHERE room_id=%s

        """ ,

        (

            self.room_number.get(),

            result["type_id"],

            self.status.get(),

            self.room_id.get()

        )

    )

    messagebox.showinfo(

        "Updated",

        "Room Updated"

    )

    self.load_rooms()

# =====
# Delete Room
# =====

def delete_room(self):

    db.execute(

```

```

        """

        DELETE FROM rooms

        WHERE room_id=%s

        """,

        (

            self.room_id.get(),

        )

    )

    messagebox.showinfo(

        "Deleted",

        "Room Deleted"

    )

    self.load_rooms()

    self.clear()

# =====
# Clear
# =====

def clear(self):

    self.room_id.set("")

    self.room_number.set("")

    self.room_type.set("")

```

```
self.price.set("")  
self.status.set("")
```

Step 2: Connect Dashboard Room Button

Open `dashboard.py`.

Replace:

```
def open_rooms(self):  
    messagebox.showinfo(  
        "Rooms",  
        "Room module will be connected later"  
    )
```

with:

```
def open_rooms(self):  
    from room import Room  
    Room()
```

Test

Run:

```
python main.py
```

Login:

```
admin  
admin123
```

Click:

Rooms

You should see:

- Room form
- Room type dropdown
- Status dropdown
- Room table
- Add / Update / Delete functions

Next: Part 6 — Booking Management Module

The booking module will add:

- ✓ Select customer
- ✓ Select available room
- ✓ Check-in date
- ✓ Check-out date
- ✓ Number of guests
- ✓ Automatic room status update
- ✓ Booking history
- ✓ Cancel booking

Great. We will now create **Part 6: Booking Management Module**.

This module will connect:

- Customer Management
- Room Management
- Booking Database

Features included:

- ✓ Select customer
- ✓ Select available room
- ✓ Check-in date
- ✓ Check-out date

- ✓ Number of adults/children
 - ✓ Create booking
 - ✓ View bookings
 - ✓ Cancel booking
 - ✓ Automatically change room status to "Occupied"
-

Step 1: Create `booking.py`

Create:

HotelManagementSystem/

|— `booking.py` <-- create this file

Add the following code:

```
import ttkbootstrap as ttk

from ttkbootstrap.constants import *

from tkinter import messagebox

from datetime import date

from database import db
```

```
class Booking:
```

```
    def __init__(self):

        self.window = ttk.Toplevel()

        self.window.title(
            "Booking Management"
        )

        self.window.geometry(
```

```
        "1100x650"
    )

    # Variables

    self.booking_id = ttk.StringVar()

    self.customer = ttk.StringVar()

    self.room = ttk.StringVar()

    self.check_in = ttk.StringVar()

    self.check_out = ttk.StringVar()

    self.adults = ttk.StringVar()

    self.children = ttk.StringVar()


    # Title

    ttk.Label(

        self.window,

        text="BOOKING MANAGEMENT",

        font=("Arial",22,"bold")

    ).pack(

        pady=15

    )


    # Form

    form = ttk.Frame(

        self.window

    )
```



```
form.pack(  
    pady=10  
)  
  
# Customer  
  
ttk.Label(  
    form,  
    text="Customer"  
) .grid(  
    row=0,  
    column=0,  
    padx=10,  
    pady=10  
)  
  
self.customer_box = ttk.Combobox(  
    form,  
    textvariable=self.customer,  
    width=30  
)  
  
self.customer_box.grid(  
    row=0,  
    column=1
```

```
)
```

```
# Room
```

```
ttk.Label(
```

```
    form,
```

```
    text="Room"
```

```
).grid(
```

```
    row=0,
```

```
    column=2,
```

```
    padx=10
```

```
)
```

```
self.room_box = ttk.Combobox(
```

```
    form,
```

```
    textvariable=self.room,
```

```
    width=20
```

```
)
```

```
self.room_box.grid(
```

```
    row=0,
```

```
    column=3
```

```
)
```

```
# Dates

ttk.Label(
    form,
    text="Check In"
).grid(
    row=1,
    column=0,
    pady=10
)

ttk.Entry(
    form,
    textvariable=self.check_in
).grid(
    row=1,
    column=1
)

ttk.Label(
    form,
    text="Check Out"
).grid(
    row=1,
```

```
        column=2
    )

    ttk.Entry(
        form,
        textvariable=self.check_out
    ).grid(
        row=1,
        column=3
    )
```

```
# Guests
```

```
ttk.Label(
    form,
    text="Adults"
).grid(
    row=2,
    column=0
)
```

```
ttk.Entry(
    form,
    textvariable=self.adults
).grid(
```

```
        row=2,  
        column=1  
    )
```

```
    ttk.Label(  
        form,  
        text="Children"  
    ).grid(  
        row=2,  
        column=2  
    )
```

```
    ttk.Entry(  
        form,  
        textvariable=self.children  
    ).grid(  
        row=2,  
        column=3  
    )
```

```
# Buttons
```

```
buttons=ttk.Frame(
```

```
        self.window
    )

    buttons.pack(
        pady=10
    )

    ttk.Button(
        buttons,
        text="Book Room",
        bootstyle="success",
        command=self.create_booking
    ).grid(
        row=0,
        column=0,
        padx=10
    )

    ttk.Button(
        buttons,
        text="Cancel Booking",
        bootstyle="danger",
        command=self.cancel_booking
    ).grid(
```

```
        row=0,

        column=1,

        padx=10
    )

    ttk.Button(

        buttons,

        text="Clear",

        command=self.clear

    ).grid(

        row=0,

        column=2

    )
```

```
# Table
```

```
columns=(

    "ID",

    "Customer",

    "Room",

    "Check In",

    "Check Out",

    "Adults",

    "Children",
```

```

        "Status"
    )

    self.table=ttk.Treeview(
        self.window,
        columns=columns,
        show="headings"
    )

    for col in columns:

        self.table.heading(
            col,
            text=col
        )

        self.table.column(
            col,
            width=130
        )

    self.table.pack(
        fill=BOTH,
        expand=True,

```



```

        padx=20,

        pady=20

    )

    self.table.bind(

        "<ButtonRelease-1>",

        self.select_booking

    )

    self.load_customers()

    self.load_rooms()

    self.load_bookings()


# =====
# Load Customers
# =====

def load_customers(self):

    db.execute(

        """

        SELECT customer_id,customer_name

        FROM customers

        """

    )

```

```

rows=db.fetchall()

data=[]

for row in rows:
    data.append(
        f"{row['customer_id']} - {row['customer_name']}"
    )

self.customer_box["values"]=data


# =====
# Load Available Rooms
# =====

def load_rooms(self):

    db.execute(
        """
        SELECT room_id,room_number
        FROM rooms
        WHERE status='Available'
        """
    )

    rows=db.fetchall()

    data=[]

```

```

for row in rows:

    data.append(

        f"{row['room_id']} - {row['room_number']}"

    )

self.room_box["values"]=data


# =====
# Create Booking
# =====


def create_booking(self):

    customer_id=self.customer.get().split("-")[0]

    room_id=self.room.get().split("-")[0]


    sql="""

    INSERT INTO bookings

    (

    customer_id,

    room_id,

    check_in,

    check_out,

    adults,

```

```
children,  
booking_date,  
status  
)  
VALUES (%s, %s, %s, %s, %s, %s, %s, %s)  
"""
```

```
db.execute(  
    sql,  
    (  
        customer_id,  
        room_id,  
        self.check_in.get(),  
        self.check_out.get(),  
        self.adults.get(),  
        self.children.get(),  
        date.today(),  
        "Booked"  
    )  
)
```

```
# Update room status
```

```
db.execute(  
    sql,
```

```

        """

        UPDATE rooms

        SET status='Occupied'

        WHERE room_id=%s

        """,

        (

            room_id,

        )

    )

    messagebox.showinfo(

        "Success",

        "Room Booked Successfully"

    )

    self.load_rooms()

    self.load_bookings()

    self.clear()

# =====
# Load Bookings
# =====

def load_bookings(self):

    self.table.delete(

```

```
        *self.table.get_children()
    )
```

```
db.execute(
    """
    SELECT
    bookings.booking_id,
    customers.customer_name,
    rooms.room_number,
    bookings.check_in,
    bookings.check_out,
    bookings.adults,
    bookings.children,
    bookings.status

    FROM bookings

    JOIN customers
    ON bookings.customer_id=
    customers.customer_id

    JOIN rooms
    ON bookings.room_id=
    rooms.room_id
    """
)
```

```
)
```

```
rows=db.fetchall()
```

```
for row in rows:
```

```
    self.table.insert(
```

```
        "",
```

```
        END,
```

```
        values=(
```

```
            row["booking_id"],
```

```
            row["customer_name"],
```

```
            row["room_number"],
```

```
            row["check_in"],
```

```
            row["check_out"],
```

```
            row["adults"],
```

```
            row["children"],
```

```
            row["status"]
```

```
        )
```

```
    )
```

```
# =====
```

```
# Select Booking
```

```
# =====
```

```

def select_booking(self,event):

    item=self.table.focus()

    row=self.table.item(item) ["values"]

    if row:

        self.booking_id.set(row[0])


# =====
# Cancel Booking
# =====


def cancel_booking(self):

    if self.booking_id.get()=="":

        return


    db.execute(

        """

        UPDATE bookings

        SET status='Cancelled'

        WHERE booking_id=%s

        """,

        (

            self.booking_id.get(),

        )

```



```

    )

    messagebox.showinfo(

        "Cancelled",

        "Booking Cancelled"

    )

    self.load_bookings()

# =====
# Clear
# =====

def clear(self):

    self.customer.set("")

    self.room.set("")

    self.check_in.set("")

    self.check_out.set("")

    self.adults.set("")

    self.children.set("")

```

Step 2: Connect Dashboard Booking Button

In dashboard.py, replace:

```

def open_booking(self):

    messagebox.showinfo(
        "Bookings",

```

```
        "Booking module will be connected later"  
    )
```

with:

```
def open_booking(self):  
    from booking import Booking  
    Booking()
```

Test

Run:

```
python main.py
```

Login:

```
Username: admin  
Password: admin123
```

Go to:

```
Dashboard → Bookings
```

You should now be able to:

- Select a customer
 - Select available room
 - Enter dates
 - Book room
 - Room status changes to Occupied
 - View booking list
 - Cancel booking
-

Next: Part 7 — Check-out & Billing Module

Next module will add:

- ✓ Checkout customer
- ✓ Calculate number of days stayed
- ✓ Room charge calculation
- ✓ GST calculation
- ✓ Payment entry
- ✓ Generate invoice data
- ✓ Change room status back to Available

Great. We will now create **Part 7: Check-out & Billing Module**.

This module will handle:

Features

- ✓ View active bookings
- ✓ Select booking
- ✓ Calculate stay duration
- ✓ Calculate room charges
- ✓ GST calculation
- ✓ Record payment
- ✓ Checkout customer
- ✓ Change room status back to Available
- ✓ Store payment details in MySQL

Step 1: Create `billing.py`

Create:

HotelManagementSystem/

|— billing.py <-- create this file

Add this code:

```
import ttkbootstrap as ttk

from ttkbootstrap.constants import *

from tkinter import messagebox

from datetime import datetime

from database import db


class Billing:

    def __init__(self):

        self.window = ttk.Toplevel()

        self.window.title(
            "Checkout & Billing"
        )

        self.window.geometry(
            "1100x650"
        )

        # Variables

        self.booking_id = ttk.StringVar()

        self.customer = ttk.StringVar()

        self.room = ttk.StringVar()

        self.check_in = ttk.StringVar()
```

```
self.check_out = ttk.StringVar()
self.amount = ttk.StringVar()
self.gst = ttk.StringVar()
self.total = ttk.StringVar()
self.payment_mode = ttk.StringVar()
```

```
# Title
```

```
ttk.Label(
    self.window,
    text="CHECKOUT & BILLING",
    font=("Arial",22,"bold")
).pack(
    pady=15
)
```

```
# Form
```

```
form = ttk.Frame(
    self.window
)
form.pack(
    pady=10
)
```

```
fields=[
```

```

        ("Booking ID",self.booking_id),

        ("Customer",self.customer),

        ("Room",self.room),

        ("Check In",self.check_in),

        ("Check Out",self.check_out),

        ("Amount",self.amount),

        ("GST",self.gst),

        ("Total",self.total)

    ]

    for i,(label,var) in enumerate(fields):

        ttk.Label(

            form,

            text=label

        ).grid(

            row=i//2,

            column=(i%2)*2,

            padx=10,

            pady=8

        )

        ttk.Entry(

            form,

            textvariable=var,

```

```
        width=30

    ).grid(

        row=i//2,

        column=(i%2)*2+1

    )

ttk.Label(

    form,

    text="Payment Mode"

).grid(

    row=4,

    column=2

)

ttk.Combobox(

    form,

    textvariable=self.payment_mode,

    values=[

        "Cash",

        "Card",

        "UPI",

        "Online"

    ]

).grid(
```

```
        row=4,

        column=3

    )

# Buttons

    ttk.Button(

        self.window,

        text="Calculate Bill",

        bootstyle="info",

        command=self.calculate_bill

    ).pack(

        pady=5

    )

    ttk.Button(

        self.window,

        text="Checkout",

        bootstyle="success",

        command=self.checkout

    ).pack(

        pady=5

    )
```



```
# Table

columns=(

    "Booking ID",

    "Customer",

    "Room",

    "Check In",

    "Check Out"

)

self.table=ttk.Treeview(

    self.window,

    columns=columns,

    show="headings"

)

for col in columns:

    self.table.heading(

        col,

        text=col

    )

    self.table.column(

        col,

        width=160

    )
```

```

self.table.pack(

    fill=BOTH,

    expand=True,

    padx=20,

    pady=20

)

self.table.bind(

    "<ButtonRelease-1>",

    self.select_booking

)

self.load_active_bookings()

# =====
# Load Active Bookings
# =====

def load_active_bookings(self):

    self.table.delete(

        *self.table.get_children()

    )

    db.execute(

```

```

        """

        SELECT

        bookings.booking_id,

        customers.customer_name,

        rooms.room_number,

        bookings.check_in,

        bookings.check_out

        FROM bookings

        JOIN customers

        ON bookings.customer_id=

        customers.customer_id

        JOIN rooms

        ON bookings.room_id=

        rooms.room_id

        WHERE bookings.status='Booked'

        """

    )

    rows=db.fetchall()

    for row in rows:

        self.table.insert(

```

```

        "",
        END,
        values=(
            row["booking_id"],
            row["customer_name"],
            row["room_number"],
            row["check_in"],
            row["check_out"]
        )
    )
)

```

```

# =====
# Select Booking
# =====

```

```

def select_booking(self,event):

    item=self.table.focus()

    row=self.table.item(item) ["values"]

    if row:

        self.booking_id.set(row[0])

        self.customer.set(row[1])

        self.room.set(row[2])

        self.check_in.set(row[3])

```

```

        self.check_out.set(row[4])

# =====
# Calculate Bill
# =====

def calculate_bill(self):

    try:

        start=datetime.strptime(

            str(self.check_in.get()),

            "%Y-%m-%d"

        )

        end=datetime.strptime(

            str(self.check_out.get()),

            "%Y-%m-%d"

        )

        days=(end-start).days

        if days<=0:

            days=1

        # Get Room Price

        db.execute(

```

```

        """

        SELECT

        room_types.price

        FROM rooms

        JOIN room_types

        ON rooms.type_id=

        room_types.type_id

        WHERE rooms.room_number=%s

        """,

        (

            self.room.get(),

        )

    )

    result=db.fetchone()

    price=float(result["price"])

    amount=price*days

    gst=amount*0.18

    total=amount+gst

```

```

        self.amount.set(
            round(amount,2)
        )

        self.gst.set(
            round(gst,2)
        )

        self.total.set(
            round(total,2)
        )

    except Exception as e:

        messagebox.showerror(
            "Error",
            str(e)
        )

# =====
# Checkout
# =====

def checkout(self):

    if self.booking_id.get()=="":

```

```
messagebox.showwarning(  
    "Select",  
    "Select Booking"  
)  
  
return
```

```
# Payment Entry
```

```
db.execute(  
    """  
  
    INSERT INTO payments  
  
    (  
  
    booking_id,  
  
    amount,  
  
    gst,  
  
    total,  
  
    payment_mode,  
  
    payment_date  
  
    )  
  
    VALUES (%s,%s,%s,%s,%s,CURDATE())  
  
    """,  
  
    (  
  
        self.booking_id.get(),  
  
        self.amount.get(),
```



```
        self.gst.get(),
        self.total.get(),
        self.payment_mode.get()
    )
)
```

```
# Update booking
```

```
db.execute(
    """
    UPDATE bookings
    SET status='Completed'
    WHERE booking_id=%s
    """,
    (
        self.booking_id.get(),
    )
)
```

```
# Make room available
```

```
db.execute(
    """
    UPDATE rooms
```

```
        SET status='Available'

        WHERE room_number=%s

        """,

        (

            self.room.get(),

        )

    )

    messagebox.showinfo(

        "Success",

        "Checkout Completed"

    )

    self.load_active_bookings()
```

Step 2: Connect Billing Button

Open dashboard.py.

Replace:

```
def open_billing(self):

    messagebox.showinfo(

        "Billing",

        "Billing module will be connected later"

    )
```

with:

```
def open_billing(self):
```

```
from billing import Billing  
  
Billing()
```

Test

Run:

```
python main.py
```

Login:

```
admin  
admin123
```

Go to:

Dashboard → Billing

Workflow:

1. Create customer
2. Book room
3. Open Billing
4. Select booking
5. Click Calculate Bill
6. Complete checkout

The system will:

- Save payment
- Mark booking completed
- Free the room

Next: Part 8 — PDF Invoice Generation + Reports

Next we will add:

- ✓ Automatic invoice number
- ✓ Hotel bill format
- ✓ Customer details
- ✓ Room details
- ✓ GST breakdown
- ✓ PDF generation using ReportLab
- ✓ Print/save invoice
- ✓ Revenue reports

Great. We will now create **Part 8: PDF Invoice Generation + Reports Module**.

This module will add:

Features

- ✓ Generate PDF invoice after checkout
- ✓ Professional hotel bill format
- ✓ Customer details
- ✓ Room details
- ✓ Payment details
- ✓ GST calculation display
- ✓ Revenue report
- ✓ Booking report

We will use:

```
pip install reportlab
```

Step 1: Create `invoice.py`

Create:

```
HotelManagementSystem/
```

```
|— invoice.py    <-- create this file
```

Add:

```
from reportlab.lib.pagesizes import A4
from reportlab.pdfgen import canvas
```

```
from datetime import datetime
```

```
import os
```

```
class Invoice:
```

```
    def __init__(self):
```

```
        self.folder = "reports/invoices"
```

```
        if not os.path.exists(self.folder):
```

```
            os.makedirs(self.folder)
```

```
    def generate_invoice(
```

```
        self,
```

```
        booking_id,
```

```
        customer,
```

```
        room,
```

```
        amount,
```

```
        gst,  
        total,  
        payment_mode  
    ):  
  
    filename = (  
        self.folder +  
        "/Invoice_" +  
        str(booking_id) +  
        ".pdf"  
    )  
  
    pdf = canvas.Canvas(  
        filename,  
        pagesize=A4  
    )  
  
    width,height=A4  
  
    # Header  
  
    pdf.setFont(  
        "Helvetica-Bold",  
        20  
    )
```

```
pdf.drawCentredString(  
    width/2,  
    height-50,  
    "HOTEL MANAGEMENT SYSTEM"  
)
```

```
pdf.setFont(  
    "Helvetica",  
    12  
)
```

```
pdf.drawCentredString(  
    width/2,  
    height-80,  
    "Customer Invoice"  
)
```

```
y=height-130
```

```
details=[  
  
    ("Invoice No",booking_id),  
  
    ("Date",  
        datetime.now().strftime("%d-%m-%Y")),
```

```
        ("Customer",customer),

        ("Room Number",room),

        ("Amount",amount),

        ("GST 18%",gst),

        ("Total Amount",total),

        ("Payment Mode",payment_mode)

    ]
```

```
for label,value in details:
```

```
    pdf.drawString(

        80,

        y,

        str(label)

    )
```

```
    pdf.drawString(

        250,

        y,

        str(value)

    )
```

```
y-=30
```



```
pdf.line(  
    70,  
    y,  
    500,  
    y  
)  
  
y-=40  
  
pdf.drawString(  
    80,  
    y,  
    "Thank you for choosing our hotel."  
)  
  
pdf.save()  
  
return filename
```

Step 2: Update `billing.py`

Open:

`billing.py`

Add at the top:

```
from invoice import Invoice
```

Find the checkout function.

After:

```
messagebox.showinfo(  
    "Success",  
    "Checkout Completed"  
)
```

Add:

```
invoice = Invoice()  
  
file = invoice.generate_invoice(  
    self.booking_id.get(),  
    self.customer.get(),  
    self.room.get(),  
    self.amount.get(),  
    self.gst.get(),  
    self.total.get(),  
    self.payment_mode.get()  
)  
  
messagebox.showinfo(  
    "Invoice Generated",  
    f"Invoice saved:\n{file}"  
)
```

Now every checkout will generate:

```
reports/  
└─ invoices/  
    └─ Invoice_1.pdf
```

Step 3: Create Reports Module

Create:

```
reports.py
```

Add:

```
import ttkbootstrap as ttk  
  
from ttkbootstrap.constants import *  
  
from database import db
```

```
class Reports:  
  
    def __init__(self):  
  
        self.window = ttk.Toplevel()  
  
        self.window.title(  
            "Reports"  
        )  
  
        self.window.geometry(  
            "900x600"  
        )
```

```
ttk.Label(  
    self.window,  
    text="HOTEL REPORTS",  
    font=(  
        "Arial",  
        22,  
        "bold"  
    )  
) .pack(  
    pady=20  
)  
  
self.text=ttk.Text(  
    self.window,  
    width=100,  
    height=30  
)  
  
self.text.pack(  
    padx=20,  
    pady=20  
)  
  
self.load_reports()
```

```
def load_reports(self):

    self.text.delete(

        "1.0",

        "end"

    )

    # Customers

    db.execute(

        "SELECT COUNT(*) total FROM customers"

    )

    customers=db.fetchone()["total"]

    # Rooms

    db.execute(

        "SELECT COUNT(*) total FROM rooms"

    )

    rooms=db.fetchone()["total"]

    # Revenue
```

```

db.execute(
    """
    SELECT SUM(total) revenue
    FROM payments
    """
)

result=db.fetchone()

revenue=result["revenue"]

if revenue is None:
    revenue=0

report=f"""
HOTEL MANAGEMENT REPORT

-----

Total Customers : {customers}

Total Rooms      : {rooms}

Total Revenue    : ₹ {revenue}

Generated Report Successfully
"""

```

```
self.text.insert(  
    "end",  
    report  
)
```

Step 4: Connect Reports Button

Open:

dashboard.py

Replace:

```
def open_reports(self):  
    messagebox.showinfo(  
        "Reports",  
        "Reports module will be connected later"  
    )
```

with:

```
def open_reports(self):  
    from reports import Reports  
    Reports()
```

Test

Run:

python main.py

Flow:

1. Login
2. Create customer
3. Book room
4. Checkout
5. Invoice PDF will be generated
6. Open Reports

Current Completed Modules

- ✓ Database
- ✓ Login
- ✓ Dashboard
- ✓ Customer Management
- ✓ Room Management
- ✓ Booking Management
- ✓ Checkout & Billing
- ✓ PDF Invoice
- ✓ Reports

Next: Part 9 — Employee Management + Final Integration

Next we will add:

- ✓ Employee CRUD
- ✓ Salary management
- ✓ Search employee
- ✓ Dashboard integration
- ✓ Final project cleanup

- ✓ Error handling improvements
- ✓ Complete running instructions

Great. We will now complete **Part 9: Employee Management + Final Integration**.

This module adds:

Employee Management Features

- ✓ Add employee
- ✓ Update employee
- ✓ Delete employee
- ✓ View employee list
- ✓ Search employee
- ✓ Salary management
- ✓ Connect with Dashboard

After this, we will also make some final integration improvements.

Step 1: Create `employee.py`

Create:

```
HotelManagementSystem/
```

```
|— employee.py  <-- create this file
```

Add:

```
import ttkbootstrap as ttk  
  
from ttkbootstrap.constants import *
```

```
from tkinter import messagebox

from database import db


class Employee:

    def __init__(self):

        self.window = ttk.Toplevel()

        self.window.title(
            "Employee Management"
        )

        self.window.geometry(
            "1000x600"
        )

        # Variables

        self.employee_id = ttk.StringVar()

        self.name = ttk.StringVar()

        self.designation = ttk.StringVar()

        self.salary = ttk.StringVar()

        self.mobile = ttk.StringVar()

        self.address = ttk.StringVar()

        ttk.Label(

            self.window,

            text="EMPLOYEE MANAGEMENT",

            font=("Arial",22,"bold")
```

```
) .pack(
    pady=15
)

# Form

form = ttk.Frame(
    self.window
)

form.pack()

fields=[
    ("Name", self.name),
    ("Designation", self.designation),
    ("Salary", self.salary),
    ("Mobile", self.mobile),
    ("Address", self.address)
]

for i, (label, var) in enumerate(fields):

    ttk.Label(
        form,
        text=label
    ).grid(
```

```

        row=i//2,

        column=(i%2)*2,

        padx=10,

        pady=10

    )

    ttk.Entry(

        form,

        textvariable=var,

        width=30

    ).grid(

        row=i//2,

        column=(i%2)*2+1

    )


# Buttons

btn=ttk.Frame(

    self.window

)

btn.pack(

    pady=15

)

```

```
ttk.Button(  
    btn,  
    text="Add",  
    bootstyle="success",  
    command=self.add_employee  
) .grid(  
    row=0,  
    column=0,  
    padx=10  
)
```

```
ttk.Button(  
    btn,  
    text="Update",  
    bootstyle="primary",  
    command=self.update_employee  
) .grid(  
    row=0,  
    column=1,  
    padx=10  
)
```

```
ttk.Button(  
    btn,
```

```
        text="Delete",
        bootstyle="danger",
        command=self.delete_employee
    ).grid(
        row=0,
        column=2,
        padx=10
    )
```

```
    ttk.Button(
        btn,
        text="Clear",
        command=self.clear
    ).grid(
        row=0,
        column=3
    )
```

```
# Table
```

```
columns=(
    "ID",
    "Name",
    "Designation",
    "Salary",
```

```

        "Mobile",
        "Address"
    )

    self.table=ttk.Treeview(
        self.window,
        columns=columns,
        show="headings"
    )

    for col in columns:

        self.table.heading(
            col,
            text=col
        )

        self.table.column(
            col,
            width=140
        )

    self.table.pack(
        expand=True,
        fill=BOTH,
        padx=20,

```

```

        pady=20
    )

    self.table.bind(
        "<ButtonRelease-1>",
        self.select_employee
    )

    self.load_employee()

# Add Employee
def add_employee(self):

    db.execute(
        """
        INSERT INTO employees
        (
            employee_name,
            designation,
            salary,
            mobile,
            address
        )
        VALUES (%s,%s,%s,%s,%s)
    """
    )

```



```

        """,
        (
            self.name.get(),
            self.designation.get(),
            self.salary.get(),
            self.mobile.get(),
            self.address.get()
        )
    )

    messagebox.showinfo(
        "Success",
        "Employee Added"
    )

    self.load_employee()
    self.clear()

# Load Employee

def load_employee(self):

    self.table.delete(
        *self.table.get_children()
    )

```

```

db.execute(
    "SELECT * FROM employees"
)

rows=db.fetchall()

for row in rows:

    self.table.insert(
        "",
        END,
        values=(
            row["employee_id"],
            row["employee_name"],
            row["designation"],
            row["salary"],
            row["mobile"],
            row["address"]
        )
    )

# Select

def select_employee(self,event):

```

```

item=self.table.focus()

row=self.table.item(item) ["values"]

if row:

    self.employee_id.set(row[0])

    self.name.set(row[1])

    self.designation.set(row[2])

    self.salary.set(row[3])

    self.mobile.set(row[4])

    self.address.set(row[5])

# Update

def update_employee(self):

    db.execute(

        """

        UPDATE employees SET

        employee_name=%s,

        designation=%s,

        salary=%s,

        mobile=%s,

        address=%s

```

```

WHERE employee_id=%s
"""
(
    self.name.get(),
    self.designation.get(),
    self.salary.get(),
    self.mobile.get(),
    self.address.get(),
    self.employee_id.get()
)
)

messagebox.showinfo(
    "Updated",
    "Employee Updated"
)

self.load_employee()

# Delete

def delete_employee(self):

    db.execute(
        """

```

```
        DELETE FROM employees
        WHERE employee_id=%s
        """
    )
    (
        self.employee_id.get(),
    )
)

messagebox.showinfo(
    "Deleted",
    "Employee Deleted"
)

self.load_employee()
self.clear()

# Clear

def clear(self):

    self.employee_id.set("")
    self.name.set("")
    self.designation.set("")
    self.salary.set("")
    self.mobile.set("")
```

```
self.address.set("")
```

Step 2: Connect Employee Button

Open `dashboard.py`.

Replace:

```
def open_employee(self):  
    messagebox.showinfo(  
        "Employees",  
        "Employee module will be connected later"  
    )
```

with:

```
def open_employee(self):  
    from employee import Employee  
    Employee()
```

Final Project Structure

Your project now looks like:

HotelManagementSystem/

```
|  
— main.py  
— config.py  
— database.py  
— login.py  
— dashboard.py  
  
— customer.py  
— room.py  
— booking.py  
— billing.py
```

```
├── invoice.py
├── employee.py
├── reports.py
├── database/
│   └── hotel_management.sql
├── reports/
│   └── invoices/
└── requirements.txt
```

Final Installation Steps

Install packages:

```
pip install ttkbootstrap
pip install mysql-connector-python
pip install pillow
pip install reportlab
```

Create database:

```
Open MySQL
Run:
database/hotel_management.sql
```

Edit:

```
config.py
```

Change:

```
DB_PASSWORD="your_mysql_password"
```

Run:

```
python main.py
```

Login:

```
Username: admin
```

Password: admin123

Completed Modules

- ✓ MySQL Database
 - ✓ Login System
 - ✓ Dashboard
 - ✓ Customer Management
 - ✓ Room Management
 - ✓ Booking Management
 - ✓ Checkout
 - ✓ Billing
 - ✓ GST Calculation
 - ✓ PDF Invoice
 - ✓ Reports
 - ✓ Employee Management
-

Recommended Final Improvements Before Using It as a Real Project

For a production/college final submission, the next enhancement would be:

Part 10 — Security + Professional Upgrade

It would add:

- Password hashing (bcrypt)
- Better validation
- Exception handling
- Backup/restore database

- Application icon
- Search bars
- Date picker widgets
- Better invoice formatting
- Role permissions (Admin / Receptionist)
- Project documentation

That would make it closer to a commercial hotel management application.

Yes. We will add **Part 10 — Security + Professional Upgrade**.

This upgrade will improve the existing project with:

- ✓ Password hashing using `bcrypt`
- ✓ Input validation
- ✓ Better exception handling
- ✓ Database backup & restore
- ✓ Application icon support
- ✓ Search functionality
- ✓ Date picker widgets
- ✓ Improved invoice formatting
- ✓ Admin / Receptionist roles
- ✓ Project documentation

Because this is a large upgrade, I will split it into sections so each part remains usable.

Part 10.1 — Install Additional Packages

Run:

```
pip install bcrypt
pip install tkcalendar
```

Your requirements.txt becomes:

```
mysql-connector-python
ttkbootstrap
Pillow
reportlab
bcrypt
tkcalendar
```

Part 10.2 — Secure Password Hashing

Create:

```
security.py
```

Code:

```
import bcrypt

def hash_password(password):
    password = password.encode("utf-8")

    hashed = bcrypt.hashpw(
        password,
        bcrypt.gensalt()
    )

    return hashed.decode("utf-8")

def verify_password(password, hashed):
    password = password.encode("utf-8")

    hashed = hashed.encode("utf-8")
```

```
    return bcrypt.checkpw(  
        password,  
        hashed  
    )
```

Part 10.3 — Update Users Table

Run this MySQL:

```
USE hotel_management;
```

```
ALTER TABLE users  
MODIFY password VARCHAR(255);
```

Part 10.4 — Create Secure Admin User

Create:

```
create_admin.py  
from database import db  
  
from security import hash_password  
  
username="admin"  
  
password="admin123"  
  
role="Admin"  
  
  
hashed=hash_password(password)
```

```
db.execute(

"""
INSERT INTO users
(username,password,role)

VALUES (%s,%s,%s)

""",

(
username,
hashed,
role
)

)

print("Admin Created")
```

Run:

```
python create_admin.py
```

Part 10.5 — Secure Login

Replace login checking code.

In login.py:

Add:

```
from security import verify_password
```

Replace:

```
sql="""

SELECT *
```

```
FROM users

WHERE username=%s

AND password=%s

"""
```

with:

```
sql="""

SELECT *

FROM users

WHERE username=%s

"""
```

Replace:

```
db.execute(
    sql,
    (username,password)
)
```

with:

```
db.execute(
    sql,
    (username,)
)
```

Replace:

```
if user:
```

with:

```
if user and verify_password(
    password,
    user["password"]
):
```

Now passwords are stored securely.

Part 10.6 — Role Permissions

Modify dashboard initialization:

Change:

```
def __init__(self, user):  
  
to:  
  
def __init__(self, user):  
    self.user=user  
    self.role=user["role"]
```

Add:

```
if self.role=="Receptionist":  
    employee_button["state"]="disabled"  
    reports_button["state"]="disabled"
```

Now:

- Admin → Full access
 - Receptionist → Booking, Customer, Billing only
-

Part 10.7 — Validation Utility

Create:

```
validation.py
import re

def validate_mobile(number):

    return re.match(
        r"^[0-9]{10}$",
        number
    )

def validate_email(email):

    return re.match(
        r"^[w\.-]+@[w\.-]+\.\w+$",
        email
    )

def validate_required(value):

    if value.strip()=="":
        return False

    return True
```

Example usage:

```
from validation import validate_mobile

if not validate_mobile(
    self.mobile.get()
):

    messagebox.showerror(
        "Error",
        "Invalid Mobile Number"
    )
```

```
return
```

Part 10.8 — Exception Handling Database

Update database.py:

```
def execute(self, query, values=None):  
    try:  
        if values:  
            self.cursor.execute(  
                query,  
                values  
            )  
        else:  
            self.cursor.execute(  
                query  
            )  
  
        self.connection.commit()  
  
    except Exception as e:  
  
        print(  
            "Database Error:",  
            e  
        )  
  
        self.connection.rollback()
```

Part 10.9 — Date Picker

Install:

```
pip install tkcalendar
```

Example replacement for date Entry:

```
from tkcalendar import DateEntry
```

```
checkin = DateEntry(  
    form,  
    width=15,  
    date_pattern="yyyy-mm-dd"  
)
```

```
checkin.pack()
```

Now users select dates from a calendar instead of typing.

Part 10.10 — Search Bar Example

Add to Customer module:

```
search_text=ttk.StringVar()
```

```
search_box=ttk.Entry(  
    window,  
    textvariable=search_text  
)
```

```
search_box.pack()
```

```
ttk.Button(  
    window,  
    text="Search",
```

```
command=search_customer  
).pack()
```

Function:

```
def search_customer(self):  
  
    db.execute(  
        """  
        SELECT *  
        FROM customers  
        WHERE customer_name LIKE %s  
        """,  
        (  
            "%"+search_text.get()+"%",  
        )  
    )  
  
    rows=db.fetchall()
```

Part 10.11 — Database Backup

Create:

```
backup.py  
import subprocess  
  
from datetime import datetime  
  
def backup_database():
```

```
filename=(  
    "backup_"  
    +  
    datetime.now()  
    .strftime("%Y%m%d")  
    +  
    ".sql"  
)  
  
command=f"""  
mysqldump  
  
-u root  
  
-p hotel_management  
  
> {filename}  
"""  
  
subprocess.call(  
    command,  
    shell=True  
)  
  
return filename
```

Part 10.12 — Restore Database

Create:

```
restore.py  
import subprocess
```

```
def restore_database(file):  
  
    command=f"""  
  
    mysql  
  
    -u root  
  
    -p hotel_management  
  
    < {file}  
  
    """  
  
    subprocess.call(  
        command,  
  
        shell=True  
  
    )
```

Part 10.13 — Application Icon

Add:

```
assets/  
    hotel.ico
```

In every main window:

```
self.window.iconbitmap(  
    "assets/hotel.ico"  
)
```

Part 10.14 — Improved Invoice Header

In `invoice.py`:

Add:

```
pdf.setFillColorRGB(  
    0.1,  
    0.4,  
    0.7  
)  
  
pdf.rect(  
    50,  
    height-100,  
    500,  
    50,  
    fill=1  
)  
  
pdf.setFillColorRGB(  
    1,  
    1,  
    1  
)  
  
pdf.setFont(  
    "Helvetica-Bold",  
    20  
)  
  
pdf.drawString(  
    100,  
    height-70,  
    "HOTEL MANAGEMENT SYSTEM"  
)
```

This creates a professional colored invoice header.

Part 10.15 — Documentation

Create:

`README.md`

Content:

`# Hotel Management System`

`## Technology`

`Python 3`

`Tkinter GUI`

`MySQL Database`

`## Features`

- `- Login System`
- `- Role Based Access`
- `- Customer Management`
- `- Room Management`
- `- Booking`
- `- Checkout`
- `- Billing`
- `- GST Invoice`
- `- Reports`
- `- Employee Management`

`## Installation`

Install packages:

`pip install -r requirements.txt`

```
## Database Setup
```

```
Import:
```

```
database/hotel_management.sql
```

```
## Run
```

```
python main.py
```

```
## Default Login
```

```
Username:  
admin
```

```
Password:  
admin123
```

After Part 10 Upgrade Your Project Has:

- ✓ Secure authentication
- ✓ Role management
- ✓ Better data validation
- ✓ Backup system
- ✓ Restore system
- ✓ Better invoice
- ✓ Better user experience
- ✓ Documentation

The remaining optional professional features would be:

Part 11 — Final Commercial Upgrade

- Dashboard charts
- Inventory management
- Restaurant/food billing
- SMS/email notifications
- Multi-hotel support
- Installer creation (.exe)
- Complete project report (SRS, ER diagram, UML, screenshots)

Yes, I can help you add **Part 11 — Final Commercial Upgrade**. This is the final expansion layer that turns the current hotel management system into a more commercial-style application.

Because these features are large (especially **multi-hotel support, restaurant billing, notifications, installer creation, and documentation**), I will provide them in sections.

Part 11 Modules

We will add:

1. Dashboard Analytics

- Charts using `matplotlib`
- Revenue graph
- Booking statistics
- Room occupancy chart

2. Inventory Management

- Store items
- Stock in/out
- Low-stock alerts
- Supplier details

3. Restaurant / Food Billing

- Food menu
- Orders
- Table/order management

- Restaurant invoice
- 4. SMS / Email Notification**
 - Booking confirmation
 - Checkout message
 - Email invoice sending
- 5. Multi-Hotel Support**
 - Multiple branches
 - Hotel selection
 - Separate rooms/bookings/data
- 6. Windows Installer**
 - .exe creation using PyInstaller
 - Application packaging
- 7. Complete Documentation**
 - SRS Document
 - ER Diagram
 - UML Diagrams
 - Database Design
 - User Manual
 - Screenshots checklist

Part 11.1 — Dashboard Charts

Install

```
pip install matplotlib
```

Create `charts.py`

```
import matplotlib.pyplot as plt

from database import db
```

```

class DashboardCharts:

    def revenue_chart(self):

        db.execute(
            """
            SELECT
            payment_date,
            SUM(total)
            FROM payments
            GROUP BY payment_date
            """
        )

        data=db.fetchall()

        dates=[]
        amounts=[]

        for row in data:

            dates.append(
                str(row["payment_date"])
            )

            amounts.append(
                float(row["SUM(total)"])
            )

        plt.figure(
            figsize=(8,4)
        )

        plt.plot(
            dates,
            amounts,
            marker="o"
        )

```

```
plt.title(  
    "Hotel Revenue"  
)
```

```
plt.xlabel(  
    "Date"  
)
```

```
plt.ylabel(  
    "Revenue"  
)
```

```
plt.xticks(  
    rotation=45  
)
```

```
plt.show()
```

```
def booking_chart(self):
```

```
    db.execute(  
        """  
        SELECT  
        status,  
        COUNT(*)  
        FROM bookings  
        GROUP BY status  
        """  
    )
```

```
    rows=db.fetchall()
```

```
    labels=[]
```

```
    values=[]
```

```
    for row in rows:
```

```
        labels.append(  
            row["status"]  
        )  
  
        values.append(  
            row["COUNT (*)"]  
        )  
  
plt.pie(  
  
    values,  
  
    labels=labels,  
  
    autopct="%1.1f%%"  
  
)  
  
plt.title(  
    "Booking Status"  
)  
  
plt.show()
```

Add Dashboard Buttons

In `dashboard.py`:

```
from charts import DashboardCharts
```

Add:

```
def open_charts(self):  
    chart=DashboardCharts()  
  
    chart.revenue_chart()
```

Part 11.2 — Inventory Management

SQL Table

Add to:

```
hotel_management.sql
```

```
CREATE TABLE inventory(  
    item_id INT AUTO_INCREMENT PRIMARY KEY,  
    item_name VARCHAR(100),  
    category VARCHAR(50),  
    quantity INT,  
    price DECIMAL(10,2),  
    supplier VARCHAR(100)  
);
```

Create `inventory.py`

```
import ttkbootstrap as ttk  
from tkinter import messagebox  
from database import db  
  
class Inventory:  
  
    def __init__(self):  
        self.win=ttk.Toplevel()  
        self.win.title()
```

```
        "Inventory Management"
    )

    self.win.geometry(
        "900x600"
    )

    self.name=ttk.StringVar()

    self.category=ttk.StringVar()

    self.qty=ttk.StringVar()

    self.price=ttk.StringVar()

    self.supplier=ttk.StringVar()


    ttk.Label(
        self.win,
        text="Inventory",
        font=("Arial",20,"bold")
    ).pack()


    for text,var in [

        ("Item",self.name),

        ("Category",self.category),

        ("Quantity",self.qty),

        ("Price",self.price),

        ("Supplier",self.supplier)

    ]:

        ttk.Entry(

            self.win,
```

```

        textvariable=var

    ).pack(

        pady=5

    )

    ttk.Button(

        self.win,

        text="Add",

        command=self.add

    ).pack()

def add(self):

    db.execute(

        """

        INSERT INTO inventory

        (

            item_name,

            category,

            quantity,

            price,

            supplier

        )

        VALUES (%s,%s,%s,%s,%s)

        """,

```

```
(  
    self.name.get(),  
    self.category.get(),  
    self.qty.get(),  
    self.price.get(),  
    self.supplier.get()  
)  
  
)  
  
messagebox.showinfo(  
    "Success",  
    "Item Added"  
)
```

Part 11.3 — Restaurant / Food Billing

Database Tables

```
CREATE TABLE food_menu(  
  
    food_id INT AUTO_INCREMENT PRIMARY KEY,  
  
    food_name VARCHAR(100),  
  
    price DECIMAL(10,2)  
  
);
```

```
CREATE TABLE food_orders(  
  
    order_id INT AUTO_INCREMENT PRIMARY KEY,
```



```
booking_id INT,  
food_id INT,  
quantity INT,  
total DECIMAL(10,2)  
);
```

Create `restaurant.py`

```
import tkinter as tk  
from database import db  
  
class Restaurant:  
  
    def __init__(self):  
        self.win=tk.Toplevel()  
  
        self.win.title(  
            "Restaurant Billing"  
        )  
  
        self.win.geometry(  
            "800x500"  
        )  
  
        tk.Label(  
            self.win,  
            text="Food Billing",  
            font=("Arial",22,"bold")  
        ).pack()
```

```
self.load_food()

def load_food(self):

    db.execute(
        "SELECT * FROM food_menu"
    )

    foods=db.fetchall()

    for food in foods:

        ttk.Label(

            self.win,

            text=f"{food['food_name']} ₹{food['price']}"

        ).pack()
```

Part 11.4 — Email Notification

Install:

```
pip install secure-smtplib
```

Create:

```
email_service.py
```

```
import smtplib
```

```
from email.message import EmailMessage
```

```
def send_email(
    receiver,
```

```
        subject,  
        message  
    ):  
  
        email=EmailMessage()  
  
        email["From"]="yourhotel@gmail.com"  
  
        email["To"]=receiver  
  
        email["Subject"]=subject  
  
        email.set_content(  
            message  
        )  
  
        server=smtplib.SMTP_SSL(  
            "smtp.gmail.com",  
            465  
        )  
  
        server.login(  
            "yourhotel@gmail.com",  
            "app_password"  
        )  
  
        server.send_message(  
            email  
        )  
  
        server.quit()
```

Part 11.5 — Multi Hotel Support

Add table:

```
CREATE TABLE hotels(  
    hotel_id INT AUTO_INCREMENT PRIMARY KEY,  
    hotel_name VARCHAR(100),  
    location VARCHAR(200),  
    phone VARCHAR(20),  
    email VARCHAR(100)  
);
```

Add hotel_id column:

```
ALTER TABLE rooms  
ADD hotel_id INT;
```

```
ALTER TABLE bookings  
ADD hotel_id INT;
```

Now one database can manage:

```
Hotel A  
├ Rooms  
├ Customers  
└ Bookings
```

```
Hotel B  
├ Rooms  
├ Customers  
└ Bookings
```

Part 11.6 — Create Windows EXE

Install:

```
pip install pyinstaller
```

Build:

```
pyinstaller --onefile --windowed main.py
```

Output:

```
dist/  
└─ main.exe
```

With icon:

```
pyinstaller ^  
--onefile ^  
--windowed ^  
--icon=assets/hotel.ico ^  
main.py
```

Part 11.7 — Final Project Documentation

Create:

```
Documentation/  
├── SRS.docx  
├── ER_Diagram.png  
├── UML_Class_Diagram.png  
├── Use_Case_Diagram.png  
├── Database_Design.docx  
├── User_Manual.docx  
└── Project_Report.pdf
```

SRS Contents

1. Introduction

Project Name: Hotel Management System

Objective:

To automate hotel operations including room reservation, customer management, billing, restaurant services, inventory and reporting.

2. Functional Requirements

Module	Function
Login	User authentication
Customer	Manage guests
Room	Manage rooms
Booking	Reservation
Billing	Invoice
Restaurant	Food orders
Inventory	Stock control
Reports	Analytics

3. Non Functional Requirements

- Security
- Performance
- Reliability
- User friendly interface

ER Diagram

Main entities:

```
USER
|
|
HOTEL
|
|
ROOM
```

|
|
BOOKING
|
|
CUSTOMER

BOOKING
|
|
PAYMENT

BOOKING
|
|
FOOD_ORDER

INVENTORY

UML Classes

Login

Dashboard

Customer

Room

Booking

Billing

Invoice

Employee

Inventory

Restaurant

Reports

At this stage the system becomes a **full-scale hotel ERP application**.

The remaining work would mainly be:

- connecting all new modules into the dashboard,
- improving UI design,
- adding more validations,
- creating screenshots,
- preparing the final college/project documentation package.

You now have the complete roadmap and source structure for a **Python + Tkinter + MySQL Hotel Management System** with:

- ✓ Secure login system
- ✓ Password hashing
- ✓ Role-based access (Admin / Receptionist)
- ✓ Customer management
- ✓ Room management
- ✓ Booking system
- ✓ Checkout and billing
- ✓ GST calculation
- ✓ PDF invoice generation
- ✓ Reports
- ✓ Employee management
- ✓ Dashboard analytics
- ✓ Inventory management
- ✓ Restaurant billing
- ✓ Email notifications
- ✓ Multi-hotel database support

- ✓ Windows .exe packaging
- ✓ SRS/UML/ER documentation structure

A good next step would be to combine all modules into one clean final project package:

```
HotelManagementSystem/  
├── main.py  
├── config.py  
├── database.py  
├── security.py  
├── modules/  
│   ├── customer.py  
│   ├── room.py  
│   ├── booking.py  
│   ├── billing.py  
│   ├── employee.py  
│   ├── inventory.py  
│   └── restaurant.py  
├── services/  
│   ├── invoice.py  
│   ├── email_service.py  
│   └── backup.py  
├── assets/  
│   └── hotel.ico  
├── database/  
│   └── hotel_management.sql  
├── reports/  
├── requirements.txt  
└── README.md
```

For a final-year project or portfolio, I would also recommend adding:

- A proper **MVC architecture**
- A .sql database export
- Sample screenshots

- A project presentation (PPT)
- A complete project report in IEEE-style format
- User manual and installation guide

You have a solid foundation for a professional desktop hotel management application. Good luck with your project development!